

การออกแบบขั้นตอนวิธี

1. การลดและเอาชนะ (Decrease-and-Conquer)
2. การแบ่งและเอาชนะ (Divide-and-Conquer)
3. การแปลงและเอาชนะ (Transform-and-Conquer)

1.การลดและเอาชนะ (Decrease-and-Conquer)

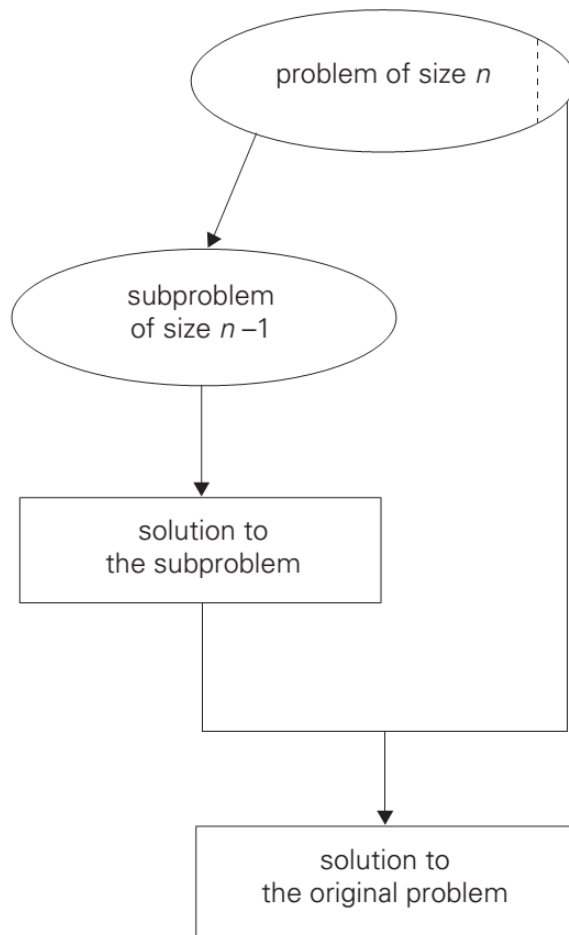
การลดและเอาชนะเป็นเทคนิคที่ใช้ประโยชน์จากความสัมพันธ์ระหว่างผลเฉลย (Solution) ของปัญหาและผลเฉลยจากการใช้จำนวนตัวอย่างทีน้อยกว่า โดยสามารถใช้ประโยชน์ได้ทั้งแบบ

1. บนลงล่าง (Top Down) ที่ใช้หลักการเรียกซ้ำ (Recursive)
2. ล่างขึ้นบน (Bottom Up) ที่ใช้หลักการวนซ้ำ (Iterative)

การลดและเอาชนะแบ่งออกเป็นสามวิธีหลัก

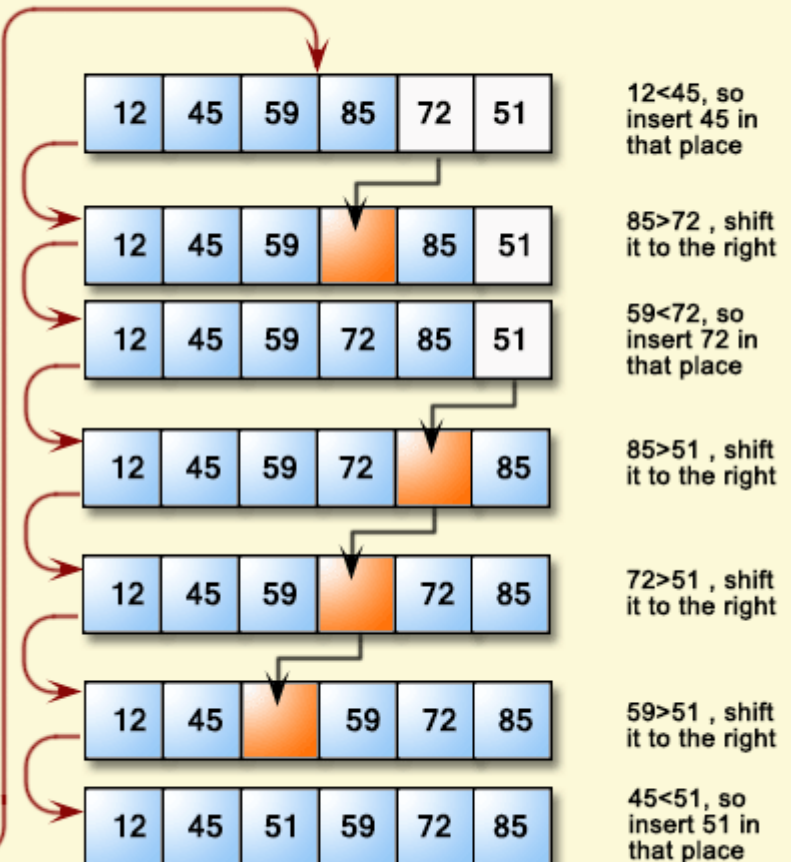
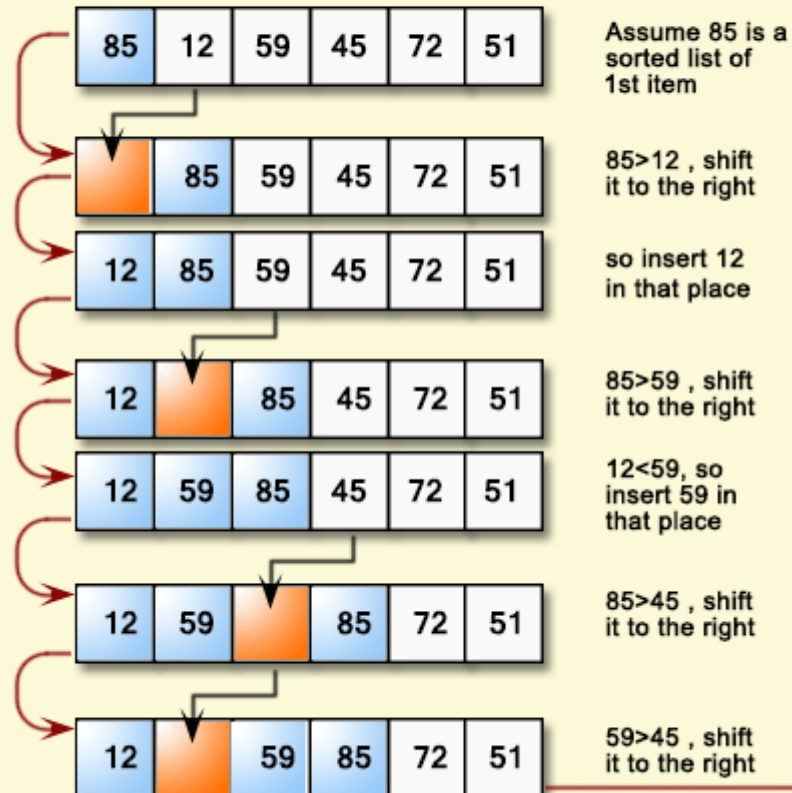
1. ลดด้วยค่าคงตัว (decrease by a constant)
2. ลดด้วยตัวประกอบคงตัว (decrease by a constant factor)
3. ลดขนาดแปรได้ (variable size decrease)

ลดด้วยค่าคงตัว (decrease by a constant)

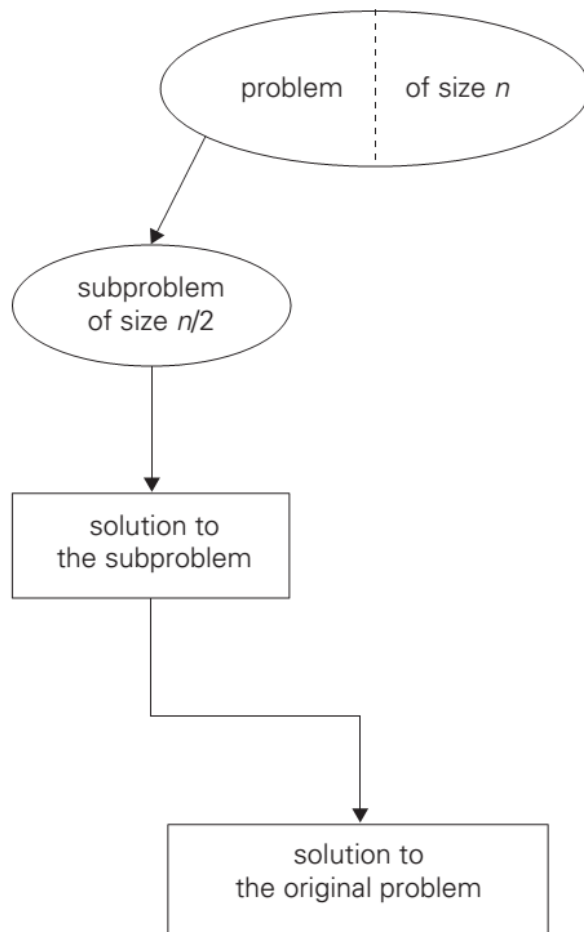


- ขนาดของตัวอย่างจะถูกลดด้วยค่าคงตัวที่เท่ากันในแต่ละการวนซ้ำของขั้นตอนวิธี
- โดยปกติค่าคงตัวจะมีค่าเท่ากับหนึ่ง เช่น การคำนวณค่ายกกำลัง
$$f(n) = \begin{cases} f(n-1) \cdot a & \text{if } n > 0, \\ 1 & \text{if } n = 0, \end{cases}$$
- คำนวณได้ทั้งบนลงล่างและล่างขึ้นบน เหมือน brute force แต่กระบวนการต่างกัน

Insertion Sort



ลดด้วยตัวประกอบคงตัว (decrease by a constant factor)

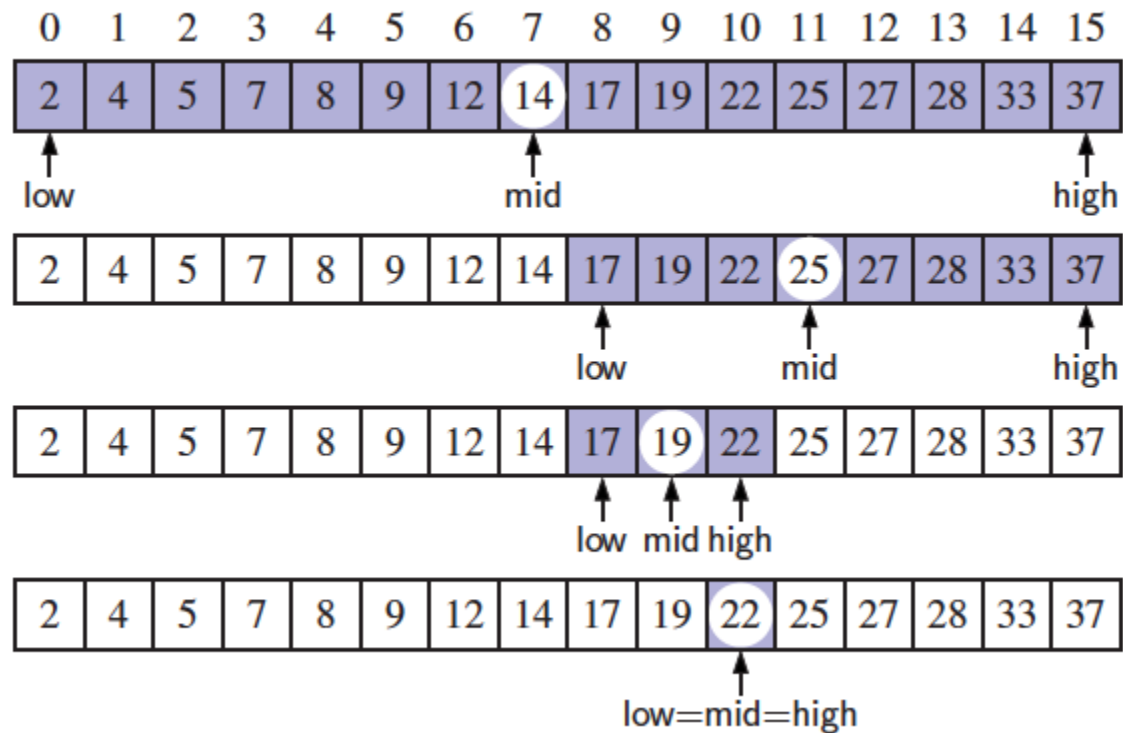


- ขนาดของตัวอย่างจะถูกลดด้วยตัวประกอบคงตัวที่เท่ากันในแต่ละการวนซ้ำของขั้นตอนวิธี
- โดยปกติตัวประกอบคงตัวจะมีค่าเท่ากับสอง เช่น การคำนวณค่ายกกำลัง

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive,} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n \text{ is odd,} \\ 1 & \text{if } n = 0. \end{cases}$$

- ขั้นตอนวิธีจะเป็น $\Theta(\log n)$

การค้นแบบทวิภาค (Binary Search)



ลดขนาดแปรได้ (variable size decrease)

- การลดขนาดจะแปรผันในแต่ละการวนซ้ำของขั้นตอนวิธี
- เช่น ขั้นตอนวิธียุคลิดสำหรับคำนวณตัวหารรวมมาก

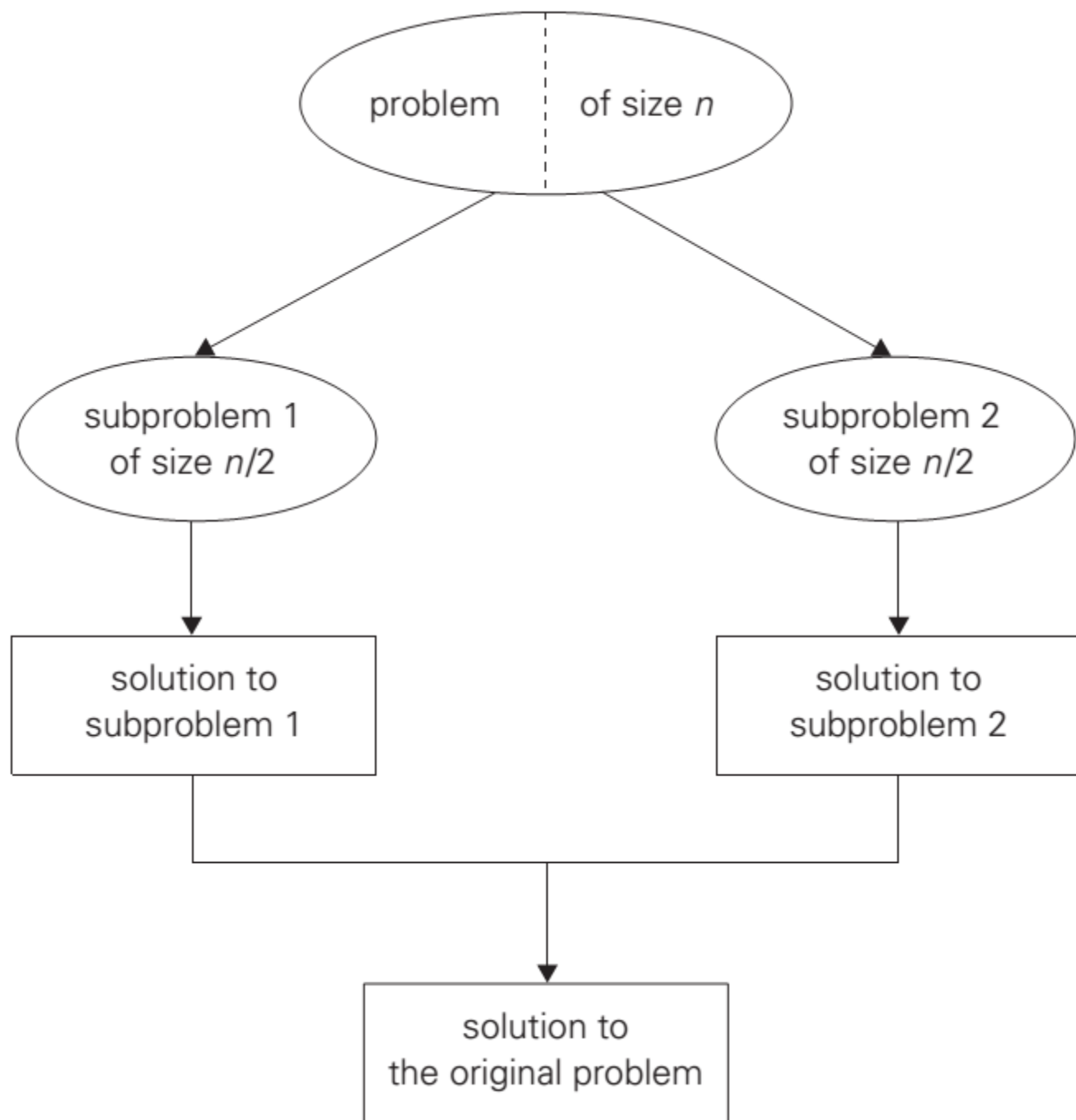
$$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$$

- ค่าของอาร์กิวเมนต์ตัวที่สองไม่ได้ลดลงด้วยค่าคงตัวและตัวประกอบคงตัว

2.การแบ่งและเอาชนะ (Divide-and-Conquer)

การแบ่งและเอาชนะเป็นเทคนิคการออกแบบขั้นตอนวิธีที่ดีที่สุดวิธีหนึ่ง โดยมีขั้นตอนดังนี้

1. แบ่งปัญหาออกเป็นปัญหาย่อยชนิดเดียวกันที่มีขนาดเท่ากัน
2. แก้ปัญหาย่อย โดยปกติใช้วิธีการเรียกซ้ำ (recursive) บางครั้งใช้ขั้นตอนวิธีที่แตกต่างกันในการแก้ปัญหาย่อยโดยเฉพาะเมื่อปัญหาย่อยมีขนาดเล็ก
3. รวมผลเฉลยที่ได้จากปัญหาย่อยเพื่อให้ได้ผลเฉลยของปัญหาดั้งเดิม



การแบ่งและเอาชนะ

การหาผลรวมของตัวเลข โดยแบ่งปัญหาเป็น 2 กรณีตัวอย่าง (instance) ที่เป็นปัญหาเดียวกัน

$$a_0 + \cdots + a_{n-1} = (a_0 + \cdots + a_{\lfloor n/2 \rfloor - 1}) + (a_{\lfloor n/2 \rfloor} + \cdots + a_{n-1}).$$

พบว่าขั้นตอนวิธีการแบ่งและเอาชนะไม่ได้มีประสิทธิภาพสูงกว่าการค้นหาคำทั้งหมดเสมอไป โดยปกติเวลาที่ใช้ในการแบ่งและเอาชนะจะน้อยกว่าเวลาที่วิธีอื่นใช้ในการแก้ปัญหา

ถึงแม้ว่าจะพิจารณาขั้นตอนวิธีแบบลำดับ (Sequential Algorithms) แต่เทคนิคนี้ยังเหมาะสำหรับการคำนวณแบบขนาน

การแบ่งและเอาชนะ

โดยทั่วไปขนาดของปัญหาจะถูกแบ่งเป็น 2 กรณีตัวอย่าง (instance) แต่ยังสามารถแบ่งขนาดของปัญหาเป็น b กรณีตัวอย่างที่มีขนาด n/b

เพื่อให้ง่ายต่อการวิเคราะห์ สมมติว่า n สามารถเขียนให้อยู่ในรูปของ ค่ายกกำลังของ b จะได้ความสัมพันธ์เวียนเกิด (Recurrence Relation)

$$T(n) = aT(n/b) + f(n),$$

เมื่อ $f(n)$ คือ เวลาที่ใช้ในการแบ่งกรณีตัวอย่างและรวมผลลัพธ์เข้าด้วยกัน

Master Theorem

Master Theorem If $f(n) \in \Theta(n^d)$ where $d \geq 0$ in recurrence (5.1), then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Analogous results hold for the O and Ω notations, too.

ความสัมพันธ์เวียนเกิด (Recurrence Relation) สำหรับการบวกเลข
ด้วยวิธีแบ่งและเอาชนะ ที่มีข้อมูลเข้าขนาด $n=2^k$

$$A(n) = 2A(n/2) + 1.$$

ดังนั้น $a = 2$, $b = 2$ และ $f(n)=1$ ดังนั้น $d = 0$ และเนื่องจาก $a > b^d$

$$A(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n).$$

การค้นหาแบบทวิภาค

และใช้วิธีการแทนค่ากลับมาหาผลเฉลยของความสัมพันธ์เวียนเกิด

$$\begin{aligned}C(n) &= C(n/2) + 1 \\&= C(n/2^2) + 1 + 1 \\&= C(n/2^3) + 1 + 1 + 1 \\&\vdots \\&= C(n/2^k = 1) + k\end{aligned}$$

แทนค่า $C(1)=1$ และ $k=\log_2 n$ จะได้

$$C(n) = 1 + \log_2 n \in O(\log n)$$

การค้นหาแบบทวิภาค $T(n) = aT(n/b) + f(n)$,

Master Theorem If $f(n) \in \Theta(n^d)$ where $d \geq 0$ in recurrence (5.1), then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Analogous results hold for the O and Ω notations, too.

และใช้วิธีการแทนค่ากลับหาผลเฉลยของความสัมพันธ์เวียนเกิด

$$C(n) = C(n/2) + 1$$

ดังนั้น $a = 1$, $b = 2$ และ $d = 0$

$$C(n) = O(n^d \log n) = O(\log n)$$

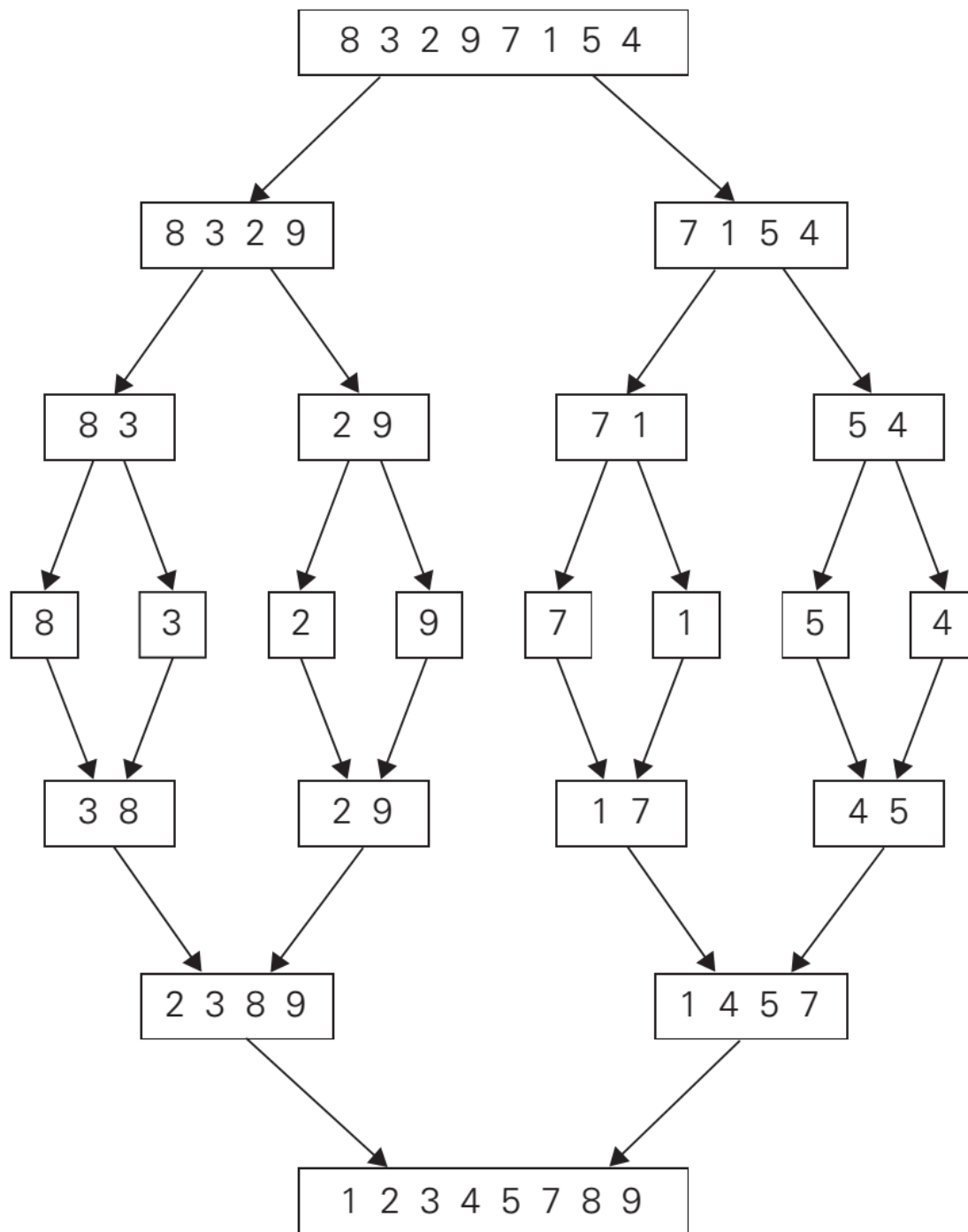
การเรียงลำดับแบบผสาน (Merge Sort)

การแบ่ง

- ถ้า S มีข้อมูลศูนย์หรือหนึ่งตัว คืนค่า S
- แต่ถ้า S มีข้อมูลอย่างน้อย 2 ตัว แบ่งข้อมูลครึ่งแรกไว้ใน $S1$ และส่วนที่เหลือไว้ใน $S2$

การเอาชนะ เรียงลำดับ $S1$ และ $S2$ ด้วยการเรียกซ้ำ

การรวม รวม $S1$ และ $S2$ ที่เรียงแล้วเข้าด้วยกัน



ประสิทธิภาพกรณีแย่ที่สุด

เพื่อให้่ายต่อการวิเคราะห์ สมมติว่า n สามารถเขียนในรูปยกกำลังของ 2 ดังนั้นความสัมพันธ์เวียนเกิดของจำนวนครั้งของการเปรียบเทียบคือ

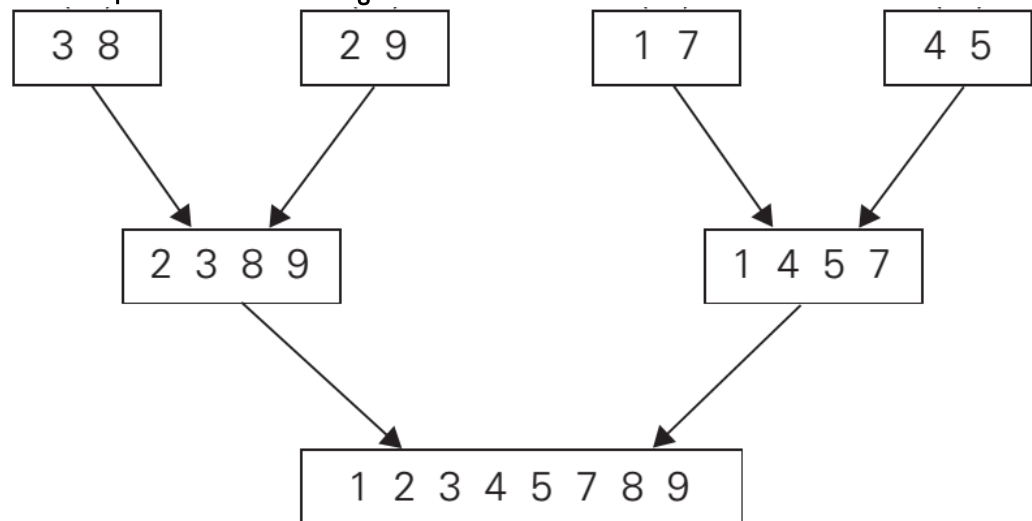
$$C(n) = 2C(n/2) + C_{merge}(n) \quad \text{for } n > 1, \quad C(1) = 0.$$

C_{merge} คือ จำนวนการเปรียบเทียบในระหว่างการผสาน

กรณีแย่สุด เมื่อค่าที่มากที่สุดสองตัวอยู่ในแถวลำดับทั้งสอง

ดังนั้น

$$C_{merge}(n) = n - 1$$



การเรียงลำดับแบบผสาน (Merge Sort)

$$T(n) = aT(n/b) + f(n),$$

Master Theorem If $f(n) \in \Theta(n^d)$ where $d \geq 0$ in recurrence (5.1), then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Analogous results hold for the O and Ω notations, too.

และใช้วิธีการแทนค่ากลับหาผลเฉลยของความสัมพันธ์เวียนเกิด

$$C(n) = 2C(n/2) + n - 1$$

ดังนั้น $a = 2$, $b = 2$ และ $d = 1$

$$C(n) = O(n^d \log n) = O(n \log n)$$

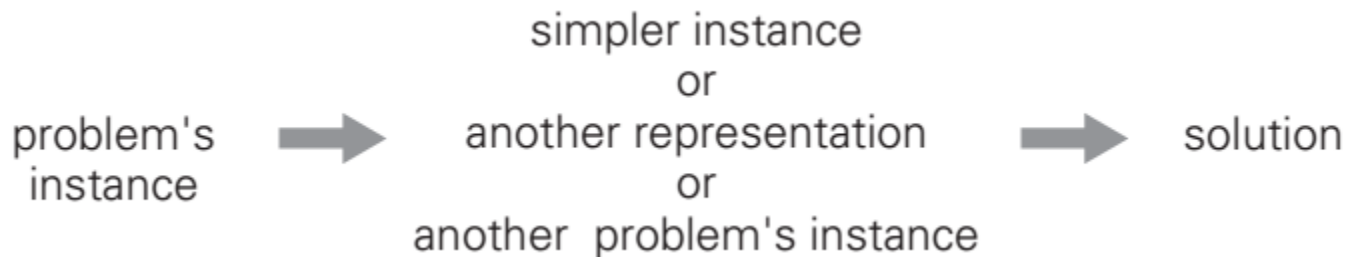
ประสิทธิภาพกรณีแย่ที่สุด

และ $C_{worst}(n) = 2C_{worst}(n/2) + n - 1$ for $n > 1$, $C_{worst}(1) = 0$.

$$C_{worst}(n) = n \log_2 n - n + 1.$$

3.การแปลงและเอาชนะ (Transform-and-Conquer)

การแปลงและเอาชนะมีขั้นตอนการทำงานสองส่วนคือ แปลงกรณีตัวอย่าง (instance) ของปัญหาให้อยู่ในรูปแบบที่แก้ได้ง่ายขึ้น จากนั้นก็แก้ปัญหานั้น โดยสามารถแบ่งออกเป็น 3 กลุ่ม คือ



การแปลงและเอาชนะ (Transform-and-Conquer)

การแปลงกรณีตัวอย่างให้ง่ายต่อการแก้ปัญหา แต่ปัญหาที่แก้ยังคงเป็นปัญหาเดิม เรียกว่า **การทำให้กรณีตัวอย่างง่ายขึ้น (Instance Simplification)** เช่น การเรียงก่อน (Presorting) เนื่องจากปัญหาหลายรูปแบบจะหาคำตอบได้ง่ายขึ้นถ้าข้อมูลถูกเรียงไว้แล้ว

การแปลงรูปแบบของกรณีตัวอย่าง เรียกว่า **การเปลี่ยนรูปแบบ (Representation Change)** เช่น การแปลงรูปแบบของต้นไม้ค้นหา

การแปลงกรณีตัวอย่างไปสู่ปัญหาที่แตกต่างจากเดิมที่มีขั้นตอนวิธีสำหรับแก้ปัญหายอยู่แล้ว หรือเรียกว่า **การลดรูปปัญหา (Problem Reduction)**

การเรียงก่อน (Presorting)

เป็นแนวความคิดที่มีมานานในวิทยาการคอมพิวเตอร์ จากข้อเท็จจริงที่ว่าปัญหาหลายประการที่เกี่ยวข้องกับรายการ (List) จะหาคำตอบได้ง่ายขึ้นถ้ารายการถูกเรียงไว้

การเรียงลำดับแบบเลือก (Selection Sort) และการเรียงลำดับแบบฟอง (Bubble Sort) และการเรียงแบบแทรก (Insertion Sort) เป็นขั้นตอนวิธีเวลากำลังสองทั้งกรณีแย่สุดและกรณีเฉลี่ย

การเรียงลำดับแบบผสาน (Mergesort) เป็นขั้นตอนวิธีเวลาล็อกเชิงเส้น และการเรียงลำดับแบบเร็ว (Quicksort) เป็นขั้นตอนวิธีเวลาล็อกเชิงเส้นสำหรับกรณีเฉลี่ยและกำลังสองสำหรับกรณีแย่สุด

ตัวอย่างที่ 1 การหาค่าข้อมูลที่มีตัวเดียวในแถวลำดับ

ขั้นตอนวิธี Brute Force ใช้การเปรียบเทียบสมาชิกของแถวลำดับจนกระทั่งเจอสมาชิกที่เหมือนกันหรือไม่มีสมาชิกให้เปรียบเทียบกันแล้วเป็นขั้นตอนวิธีเวลากำลังสองสำหรับกรณีแย่สุด

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

ตัวอย่างที่ 1 การหาค่าข้อมูลที่มีตัวเดียวในแถวลำดับ

แต่ถ้าเรียงแถวลำดับก่อนแล้วจะทำการเปรียบเทียบเฉพาะสมาชิกที่อยู่ติดกันเท่านั้น เพราะสมาชิกของแถวลำดับที่เหมือนกันจะอยู่ติดกัน

ขั้นตอนวิธีนี้จะมีการเปรียบเทียบไม่เกิน $n-1$ ครั้ง

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

ตัวอย่างที่ 1 การหาค่าข้อมูลที่มีตัวเดียวในแถวลำดับ

เวลาในการประมวลผลของขั้นตอนวิธีจะประกอบกันสองส่วน คือ เวลาที่ใช้การเรียงและเวลาที่ใช้ในการเปรียบเทียบสมาชิกที่อยู่ติดกัน ดังนั้นการเรียงจะเป็นส่วนที่ใช้ในการพิจารณาประสิทธิภาพของขั้นตอนวิธี

ถ้าเลือกใช้ขั้นตอนวิธีเรียงลำดับที่ใช้เวลาเป็นกำลังสองจะไม่มีประสิทธิภาพที่ดีกว่า Brute Force

แต่ถ้าเลือกใช้ขั้นตอนวิธีเรียงลำดับที่ดี เช่น การเรียงลำดับแบบผสาน จะทำให้ประสิทธิภาพของขั้นตอนวิธีเป็นล็อกเชิงเส้น

$$T(n) = T_{\text{sort}}(n) + T_{\text{scan}}(n) \in \Theta(n \log n) + \Theta(n) = \Theta(n \log n).$$

ตัวอย่างที่ 2 ปัญหาการค้นหา

การค้นหาแบบลำดับ (Brute Force) ในหัวข้อ 3.1 เป็นขั้นตอนวิธีเวลาเชิงเส้น แต่ถ้าแถวลำดับถูกเรียงจากนั้นสามารถใช้การค้นหาแบบทวิภาค

ALGORITHM *SequentialSearch2*($A[0..n]$, K)

//Implements sequential search with a search key as a sentinel

//Input: An array A of n elements and a search key K

//Output: The index of the first element in $A[0..n - 1]$ whose value is

// equal to K or -1 if no such element is found

$A[n] \leftarrow K$

$i \leftarrow 0$

while $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

ตัวอย่างที่ 2 ปัญหาการค้นหา

$$T(n) = T_{\text{sort}}(n) + T_{\text{search}}(n) = \Theta(n \log n) + \Theta(\log n) = \Theta(n \log n),$$

- พบว่าประสิทธิภาพแย่กว่าการค้นหาแบบลำดับ

ALGORITHM *BinarySearch*($A[0..n - 1]$, K)

//Implements nonrecursive binary search

//Input: An array $A[0..n - 1]$ sorted in ascending order and

// a search key K

//Output: An index of the array's element that is equal to K

// or -1 if there is no such element

$l \leftarrow 0$; $r \leftarrow n - 1$

while $l \leq r$ **do**

$m \leftarrow \lfloor (l + r)/2 \rfloor$

if $K = A[m]$ **return** m

else if $K < A[m]$ $r \leftarrow m - 1$

else $l \leftarrow m + 1$

return -1